2. Frequent Itemset Mining Methods

2.1 Apriori Algorithm: Finding Frequent Itemsets by Confined Candidate Generation

- The name of the algorithm is based on the fact that the algorithm uses *prior knowledge* of frequent itemset properties, as we shall see later.
- **Apriori** employs an iterative approach known as a *level-wise* search, where k-itemsets are used to explore $(k + 1)$-itemsets.
  - First, the set of frequent 1-itemsets is found by scanning the database to accumulate the count for each item, and collecting those items that satisfy minimum support. The resulting set is denoted by L_1.
  - Next, L_1 is used to find L_2, the set of frequent 2-itemsets, which is used to find L_3, and so on, until no more frequent k-itemsets can be found.
  - The finding of each L_k requires one full scan of the database.
- To improve the efficiency of the level-wise generation of frequent itemsets, an important property called the **Apriori property** is used to reduce the search space.

2. Frequent Itemset Mining Methods

2.1 Apriori Algorithm: Finding Frequent Itemsets by Confined Candidate Generation

- **Apriori property**: All nonempty subsets of a frequent itemset must also be frequent.
- How L_{k-1} is used to find L_k for $k \geq 2$. A two-step process is followed, consisting of **join** and **prune** actions.

2. Frequent Itemset Mining Methods

2.1 Apriori Algorithm: Finding Frequent Itemsets by Confined Candidate Generation

- **1. The join step.**
  - To find L_k, a set of **candidate** k-itemsets is generated by joining L_{k-1} with itself. This set of candidates is denoted C_k.
  - Let l_1 and l_2 be itemsets in L_{k-1}. The notation $l_i[j]$ refers to the jth item in l_i.
  - For efficient implementation, Apriori assumes that items within a transaction or itemset are sorted in lexicographic order. For the $(k-1)$-itemset, l_i, this means that the items are sorted such that $l_i[1] < l_i[2] < \cdots < l_i[k-1]$.
  - The join, $L_{k-1} \bowtie L_{k-1}$, is performed, where **members of L_{k-1} are joinable if their first $(k-2)$ items are in common**. That is, members l_1 and l_2 of L_{k-1} are joined if $(l_1[1] = l_2[1]) \wedge (l_1[2] = l_2[2]) \wedge \cdots \wedge (l_1[k-2] = l_2[k-2]) \wedge (l_1[k-1] < l_2[k-1])$.
    - The condition $l_1[k-1] < l_2[k-1]$ simply ensures that no duplicates are generated.
  - The resulting itemset formed by joining l_1 and l_2 is $\{l_1[1], l_1[2], ..., l_1[k-2], l_1[k-1], l_2[k-1]\}$.

2. Frequent Itemset Mining Methods
2.1 Apriori Algorithm: Finding Frequent Itemsets by Confined Candidate Generation

- **2. The prune step.**
  - C_k is a superset of L_k, that is, its members may or may not be frequent. A database scan to determine the count of each candidate in C_k would result in the determination of L_k (i.e., all candidates having a count no less than the minimum support count are frequent by definition and therefore belong to L_k).
  - C_k, however, can be huge, and so this could involve heavy computation. To reduce the size of C_k, the Apriori property is used as follows.
  - Any (k−1)-itemset that is not frequent cannot be a subset of a frequent k-itemset. Hence, if any (k−1)-subset of a candidate k-itemset is not in L_{k-1}, then the candidate cannot be frequent either and so can be removed from C_k. This **subset testing** can be done quickly by maintaining a hash tree of all frequent itemsets.

2. Frequent Itemset Mining Methods

2.1 Apriori Algorithm: Finding Frequent Itemsets by Confined Candidate Generation

- **Example 4.3. Apriori.** Let's look at a concrete example, based on the transaction database, D, of Table 4.1. There are nine transactions in this database, that is, $|D| = 9$. We use Fig. 4.2 to illustrate the Apriori algorithm for finding frequent itemsets in D.

Table 4.1 A transactional data set.

TID	List of item_IDs
T100	I1, I2, I5
T200	I2, I4
T300	I2, I3
T400	I1, I2, I4
T500	I1, I3
T600	I2, I3
T700	I1, I3
T800	I1, I2, I3, I5
T900	I1, I2, I3

2. Frequent Itemset Mining Methods

2.1 Apriori Algorithm: Finding Frequent Itemsets by Confined Candidate Generation

Table 4.1 A transactional data set.

TID	List of item_IDs
T100	I1, I2, I5
T200	I2, I4
T300	I2, I3
T400	I1, I2, I4
T500	I1, I3
T600	I2, I3
T700	I1, I3
T800	I1, I2, I3, I5
T900	I1, I2, I3

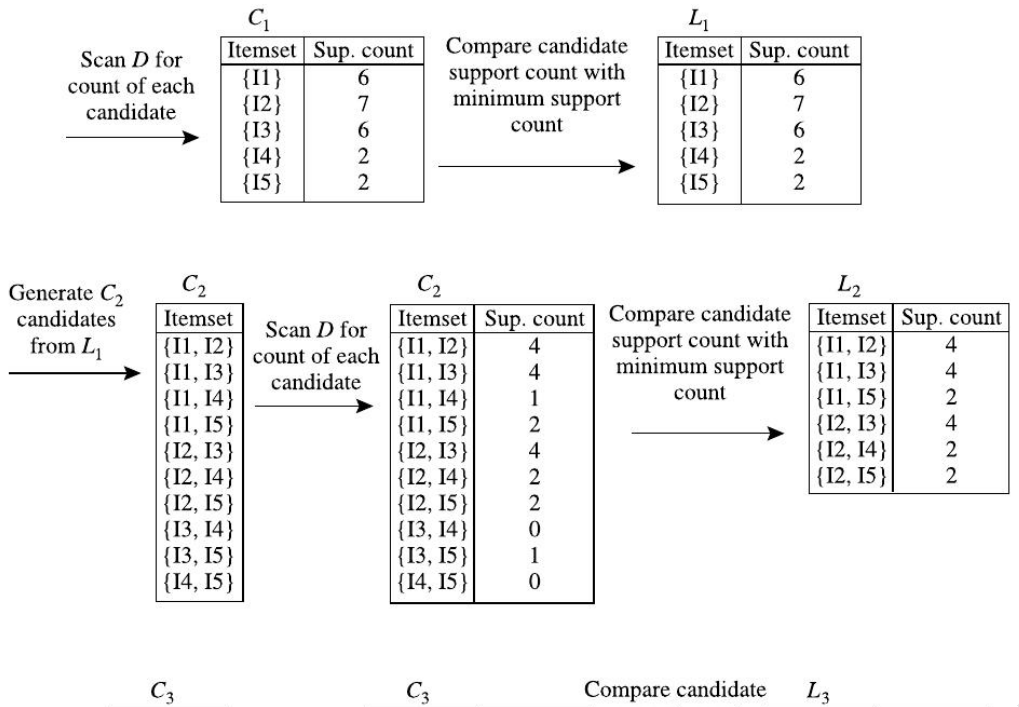

FIGURE 4.2

Generation of the candidate itemsets and frequent itemsets, where the minimum support count is 2.

- The algorithm uses $L_3 \bowtie L_3$ to generate a candidate set of 4-itemsets, C_4. Although the join results in $\{\{I_1, I_2, I_3, I_5\}\}$, itemset $\{I_1, I_2, I_3, I_5\}$ is pruned because its subset $\{I_2, I_3, I_5\}$ is not frequent.
- Thus, $C_4 = \emptyset$, and the algorithm terminates, having found all of the frequent itemsets.

2. Frequent Itemset Mining Methods

2.1 Apriori Algorithm: Finding Frequent Itemsets by Confined Candidate Generation

-

Table 4.1 A transactional data set.

TID	List of item_IDs
T100	I1, I2, I5
T200	I2, I4
T300	I2, I3
T400	I1, I2, I4
T500	I1, I3
T600	I2, I3
T700	I1, I3
T800	I1, I2, I3, I5
T900	I1, I2, I3

a. Join: $C_3 = L_2 \bowtie L_2 = \{\{I1, I2\}, \{I1, I3\}, \{I1, I5\}, \{I2, I3\}, \{I2, I4\}, \{I2, I5\}\}$
$\bowtie \{\{I1, I2\}, \{I1, I3\}, \{I1, I5\}, \{I2, I3\}, \{I2, I4\}, \{I2, I5\}\}$
$= \{\{I1, I2, I3\}, \{I1, I2, I5\}, \{I1, I3, I5\}, \{I2, I3, I4\}, \{I2, I3, I5\}, \{I2, I4, I5\}\}$.

b. Prune using the Apriori property: *all nonempty subsets of a frequent itemset must also be frequent*. Do any of the candidates have a subset that is not frequent?

- The 2-item subsets of {I1, I2, I3} are {I1, I2}, {I1, I3}, and {I2, I3}. All 2-item subsets of {I1, I2, I3} are members of L_2. Therefore, keep {I1, I2, I3} in C_3.
- The 2-item subsets of {I1, I2, I5} are {I1, I2}, {I1, I5}, and {I2, I5}. All 2-item subsets of {I1, I2, I5} are members of L_2. Therefore, keep {I1, I2, I5} in C_3.
- The 2-item subsets of {I1, I3, I5} are {I1, I3}, {I1, I5}, and {I3, I5}. {I3, I5} is not a member of L_2, and so it is not frequent. Therefore, remove {I1, I3, I5} from C_3.
- The 2-item subsets of {I2, I3, I4} are {I2, I3}, {I2, I4}, and {I3, I4}. {I3, I4} is not a member of L_2, and so it is not frequent. Therefore, remove {I2, I3, I4} from C_3.
- The 2-item subsets of {I2, I3, I5} are {I2, I3}, {I2, I5}, and {I3, I5}. {I3, I5} is not a member of L_2, and so it is not frequent. Therefore, remove {I2, I3, I5} from C_3.
- The 2-item subsets of {I2, I4, I5} are {I2, I4}, {I2, I5}, and {I4, I5}. {I4, I5} is not a member of L_2, and so it is not frequent. Therefore, remove {I2, I4, I5} from C_3.

c. Therefore, $C_3 = \{\{I1, I2, I3\}, \{I1, I2, I5\}\}$ after pruning.

FIGURE 4.3

Generation and pruning of candidate 3-itemsets, C_3, from L_2 using the Apriori property.

L_2

Itemset	Sup. count
{I1, I2}	4
{I1, I3}	4
{I1, I5}	2
{I2, I3}	4
{I2, I4}	2
{I2, I5}	2

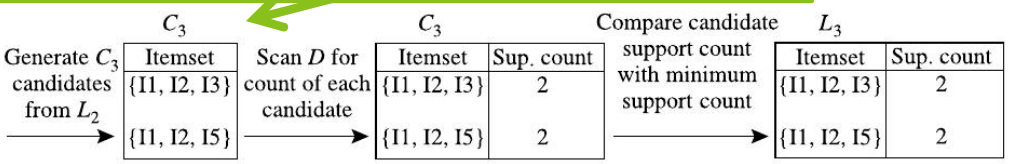

FIGURE 4.2

Generation of the candidate itemsets and frequent itemsets, where the minimum support count is 2.